

THE UNIVERSITY OF BRITISH COLUMBIA



Electrical and Computer Engineering

EECE 568: From Exploring to Building Machine Learning Models

ASSIGNMENT #2 – Neural Networks

Student Name: Navraj Singh Kambo	Student Number: 23152151	Set: _

EECE 568 - Assignment 2

Matthew Yedlin, Bhushan Gopaluni

Due Date: October 22, 2026 - 11:59 pm

Late submissions will not be accepted. In case of any medical or family emergency, please inform the instructors.

Deliverables: Please upload a ZIP file to Canvas containing a PDF for the short-answer questions and a Jupyter Notebook (.ipynb) for the coding questions.

Short-answer Questions (30 Marks)

1. Why use gradient descent for linear regression problems instead of simply applying the normal equations and a closed-form solution?
2. What issues arise from having a learning rate that is too small in gradient descent? What problems occur with a learning rate that is too large?
3. For $f(x) = x^2$, starting from $x = 2$, show how you can find the minimum of $f(x)$ using gradient descent step-by-step. Show the first two iterations using a gradient step size of 0.5.
4. Explain how we use the log-sum-exp function in L1-norm, i.e., robust regression. What is its purpose and how is this related to gradient descent?
5. Why do we use stochastic gradient descent instead of gradient descent?
6. Consider training a neural network with sigmoid activation functions and a single hidden layer, resulting in the following parameters (assuming no bias terms are used):

$$W = \begin{bmatrix} -2 & 0 & 3 \\ 2 & -2 & -1 \end{bmatrix}, v = \begin{bmatrix} 4 \\ -2 \end{bmatrix}.$$

Assuming that we are doing regression, for a training example with features $x_i^T = \begin{bmatrix} 3 & 2 & 2 \end{bmatrix}$ what are the values of the hidden layer before applying the non-linearity (z_i), hidden values after applying the non-linearity ($h(z_i)$), and the outputs ($\hat{y}_i$)?

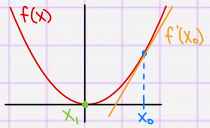
7. Is it reasonable to apply neural networks to datasets where the examples are linearly separable in the original feature space?
8. For a fully connected neural network for 3-class classification and 8 input features ($d = 8$), if the network has a hidden layer with 64 neurons ($k = 64$), how many parameters, including biases, does the network have?
9. How does increasing the dimensionality of hidden features (a hyper-parameter) enhance the performance of our classifier?
10. How does increasing the number of layers enhance the performance of our classifier?
11. Why does regularization become increasingly important as we add more layers to a neural network?
12. Neural networks often face challenges with highly non-convex objective functions. How can we mitigate this issue and improve the likelihood of finding a global optimum?
13. What is the "vanishing gradient" problem with neural networks based on sigmoid non-linearities?
14. What happens if we were to use a purely linear activation function?
15. What are the main advantages of convolutional networks over regular fully connected neural networks for image classification?

Navraj Singh Kambo
ECE 23152151
THE UNIVERSITY OF BRITISH COLUMBIA

1. For large datasets, closed form linear regression solving is more difficult (large matrix inversion). As opposed to gradient descent, which is more computationally efficient in comparison.

2. For a smaller learning rate, the training takes longer and converges much slower.
   For a larger learning rate, the algorithm can overshoot the minimum and "bounce around" or diverge.

3. $f(x)=x^2$ $x_0=2$, $\gamma=0.5$
   $f'(x)=2x$ $x_{t+1}=x_t-\gamma f'(x_t)$
   $x_1=x_0-\gamma f'(x_0)=2-(0.5)(2)(2)=0$
   $x_2=x_1-\gamma f'(x_1)=0-(0.5)(2)(0)=0$



   This converges after the first iteration

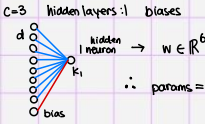
4. L_1 regression $L(w)=\sum_i^n |y_i-\hat{y}_i|=\sum_i^n |r_i|$
   L_1 regression is less sensitive to residual outliers. For example, in the set $r=\{1,2,10\}$, $r_2=10$ is an outlier. Using L_2 error, $L_2(w)=1^2+2^2+10^2=105$ vs $L_1(w)=|1|+|2|+|10|=13$. Because of this, it is more robust. However, the absolute value function is not differentiable at 0. The logsum exp function does a good job of approximating the absolute value function, while also being smooth and differentiable, which is important for gradient descent.

5. $L(w)=\frac{1}{n}\sum_i^n L_i(w) \rightarrow \nabla L(w)=\frac{1}{n}\sum_i^n \nabla L_i(w)$
   batch update: $w_{t+1}=w_t-\gamma\frac{1}{n}\sum_i^n \nabla L_i(w_t)$ every weight update requires calculating the gradient using the entire dataset (n elements).
   Stochastic gradient descent: $w_{t+1}=w_t-\gamma\nabla L_i(w_t)$ every weight update only requires calculating the gradient using a single example (i).
   SGD is faster for updating, works well with large datasets, can be updated online as new training examples are added, and has noise (random single example used in gradient calculation) which can help it converge. SGD is more noisey in comparison to batch update, which is smooth

6. $W=\begin{bmatrix}-2 & 0 & 3\\ 2 & -2 & -1\end{bmatrix}$ $v=\begin{bmatrix}4\\ -2\end{bmatrix}$ $x_i^T=\begin{bmatrix}3 & 2 & 2\end{bmatrix}$ $\sigma(z)=\frac{1}{1+e^{-z}}$
   $X \xrightarrow{W} z \xrightarrow{\sigma} h(x) \xrightarrow{v} \hat{y}$
   $\sigma(0)=\frac{1}{1+1}=\frac{1}{2}$
   $z_i=Wx_i=\begin{bmatrix}-2 & 0 & 3\\ 2 & -2 & -1\end{bmatrix}\begin{bmatrix}3\\ 2\\ 2\end{bmatrix}=\begin{bmatrix}-6+0+6\\ 6-4-2\end{bmatrix}=\begin{bmatrix}0\\ 0\end{bmatrix}$ $h(x_i)=\sigma(z_i)=\begin{bmatrix}0.5\\ 0.5\end{bmatrix}$
   $\hat{y}_i=v^T h(x_i)=v^T\sigma(z_i)=\begin{bmatrix}4 & -2\end{bmatrix}\begin{bmatrix}0.5\\ 0.5\end{bmatrix}=2-1=1$

7. If the data is linearly separable, then you can probably get away with logistic regression instead of adding in the complexity of a neural network.

8. $d=8$ $K=64$ $C=3$ hidden layers: 1 biases
   fully connected



   $\rightarrow$ $w\in\mathbb{R}^{64\times 9}$, $X\in\mathbb{R}^{8\times 1}$, $v\in\mathbb{R}^{64\times 4}$
   $\therefore$ params $=(64\times 9)+(64\times 4)=576+195=771$

9. Increasing the hiddenlayer dimensionality allows you to better represent more complex non-linear transformations and decision boundaries.

10. Make the network deeper allows you to compose non-linear functions together, where each layer could be used to abstract different features.
    Complex functions can be better represented by deep nets as opposed to wide nets with fewer layers.

11. Regularization discourages overly complex solutions, since deep networks offer a greater representational capacity and have an increased risk of overfitting.

12. There are a few different techniques such as random weight initialisation, adding momentum as a hyperparameter, adding optimisers such as ADAM, adding stochastic noise (minibatch SGD) to escape saddle points, and L2 regularization.

13. For the binary sigmoid function, it's derivative has a max value of 1/4. In a network of multiple layers, these derivatives get multiplied in a chain (chain rule), so you can think of this problem as $(\frac{1}{4})^n$. As n gets larger, $(\frac{1}{4})^n$ approaches 0, thus the vanishing gradient problem. (During back-propagation, the gradient is the product of the derivatives from output to input ex. $\frac{\partial L}{\partial h_1}=\frac{\partial L}{\partial h_3}\cdot\frac{\partial h_3}{\partial h_2}\cdot\frac{\partial h_2}{\partial h_1}$)

14. For data that isn't linearly separable, a linear activation function will not be able to separate the data for accurate representation. (XOR problem is an example)

Coding Questions (50 Marks)

Complete the sections marked with a *ToDo* tag in the jupyter notebook. We recommend using Google Colaboratory to use the available GPU power they offer. You can easily unzip the assignment2 folder, which contains a jupyter notebook and a CSV file. Upload the folder on your google drive and run the notebook.

1 Image Classification using CNNs (15 Marks)

In this question, we want to use convolutional neural networks (CNNs) to do image classification on the MNIST dataset, a collection of 28×28 images of handwritten digits from 0 to 9. Our CNN takes one image as input and predicts the digit it represents. I.e, each digit is a class. This question is mostly a tutorial on neural networks in PyTorch, which is organized into five subsections.

1.1 Dataset and Dataloader

PyTorch has an efficient and user-friendly API for loading and parsing data. Get familiar with the PyTorch dataloader as it will be an inseparable part of your assignments and projects. For clarity, the dataloaders return a python dictionary throughout the assignment.

1.2 Train, Validation, Test Split

MNIST does not provide a validation set. split the training data into a train set and a validation set. You can use the validation set for hyperparameter tuning. Use the test set only for reporting the final results.

1.3 Network

The jupyter notebook file provides two sample networks, one using a pre-defined network called ResNet, and one implemented from scratch. Try to use both of them and get familiar with neural networks in PyTorch. The networks expect a python dictionary as input and return a dictionary as output, to be consistent with other modules such as dataloaders and the training script.

1.4 Training

We have provided you with the script for training the network, calculating the loss, backpropagating the gradients, and transferring the data between CPU and GPU. Please read this part of the code thoroughly. Think about the loss function and compare it with other classification losses. Also, you can change hyperparameters like batch size and learning rate. Train the network for at least five epochs to see meaningful changes in the loss, but avoid training for too long as you have limited time and GPU resources.

1.5 Evaluation

Now we want to evaluate our trained network. We are only interested in the accuracy of the network on the test set. But for comparison, report the accuracy of the model on train, validation, and test sets. Also, plot the input images and output of your network for a few samples in the test set. You can use the function **plot_images()** provided for you in the **Helper Functions** section. Note that you should use your testing data only for evaluation, not for training or hyperparameter tuning.

2 Anomaly Detection using CNNs (15 Marks)

In this question, given three images, we want to detect the image representing a different digit. The CNN takes three different images from the MNIST dataset as inputs, where two of them have the same digit and the other one has a different digit. The network predicts the index of the image with a different digit. For example in Fig 1, the first image (from left to right) has a different digit. Therefore, the label of this sample is 0, which is the index of the first image. Note that we do not have access to the digit of each image. We only know the index of the different image. Also, in this question, we are considering only digits 0 to 4.

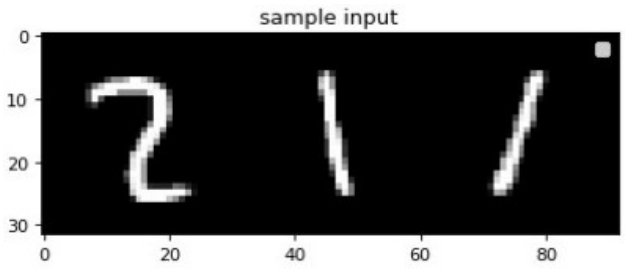


Figure 1: Sample input for question 2. The figure contains three images (from left to right): [(digit:2, index:0), (digit:1, index:1), (digit:1, index:2)]. The label for this data is 0, which is the index of the image with a different digit.

2.1 Dataset and Dataloader

The dataloader returns three randomly chosen images and the index of the image with a different digit. It is guaranteed that two and only two of the images represent the same digits. The batch size is set to 128, but feel free to change it.

2.2 Train, Validation, Test Split

Split the data into train, validation, and test sets. Print the number of samples in each dataset.

2.3 Network

Implement your CNN from scratch. **Do not use pre-defined networks.** The network should take a python dictionary as input and return a dictionary as output. More details in the jupyter notebook.

2.4 Training

The training script is provided to you similar to question 1. But you should define a suitable loss function that matches the output of your network and helps the network to learn.

2.5 Evaluation

Same as question 1, report the accuracy of the model and plot the input images and output of your network for a few samples in the test set.

3 Sentiment Analysis using RNNs (20 Marks)

The IMDB dataset contains 50000 movie reviews. Each review is labeled as either positive (1) or negative (0). We want to implement a recurrent neural network (RNN) that takes each review (series of words) as input and predicts the sentiment behind it, i.e., whether it is a positive or a negative review.

3.1 Load and Tokenize Data

We use a preprocessed version of the IMDB dataset done by Affandy Fahrizain, which is attached to your assignment folder. We then assign a unique token (number) to each word, so we can apply mathematical operations to them.

3.2 Dataloader

The dataloader returns a python dictionary containing reviews and their labels. Also, the data is split into the train, validation, and test sets.

3.3 Network

Implement your network based on Long Short-Term Memory (LSTM) blocks. The network should take a python dictionary as input and return a dictionary as output. More details in the jupyter notebook.

3.4 Training

The training script is provided to you similar to question 1. But you should define a suitable loss function that matches the output of your network and helps the network to learn. It usually takes more epochs for RNNs than CNNs to converge. Train the model for at least 10 epochs.

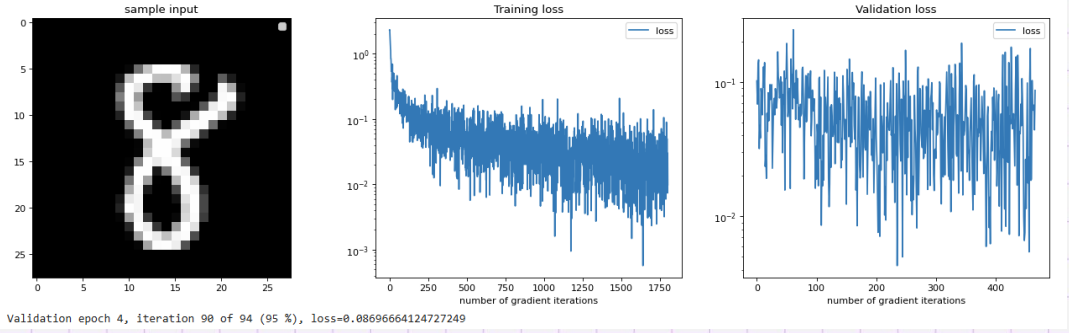
3.5 Evaluation

Report the accuracy of the model on the train, validation, and test sets. Also, print the input review and output of your network for one positive example and one negative example in the test set. Choose qualitative examples that show the effectiveness of your network.

Navraj Singh Kambo
ECE 23152151
THE UNIVERSITY OF BRITISH COLUMBIA

Part #1 results below

```
Training set size:   48000
Validation set size: 12000
Test set size:       10000
```




```
Validation epoch 4, iteration 90 of 94 (95 %), loss=0.08696664124727249
```

```
Training Accuracy:   99.57%
Validation Accuracy: 98.64%
Test Accuracy:       98.75%
```

